Deductive Databases

- **What is a deductive database system?**

A deductive database can be defined as an advanced database augmented with an inference system.

By evaluating rules against facts, new facts can be derived, which in turn can be used to answer queries. It makes a database system more powerful.

•**Some basic concepts from logic**

To understand the deductive database system well, some basic concepts from mathematical logic are needed.

- term
- n-ary predicate
- literal
- (well-formed) formula
- clause and Horn-clause
- facts
- logic program

- term

  A term is a constant, a variable or an expression of the form $f(t_1, t_2, ..., t_n)$, where $t_1, t_2, ..., t_n$ are terms and f is a function symbol.

  - Example: a, b, c, f(a, b), g(a, f(a, b)), x, y, $g(x, y)$

- n-ary predicate

  An n-ary predicate symbol is a symbol p appearing in an expression of the form $p(t_1, t_2, ..., t_n)$, called an atom, where $t_1, t_2, ..., t_n$ are terms. $p(t_1, t_2, ..., t_n)$ can only evaluate to *true* or *false*.

  - Example: p(a, b), q(a, f(a, b)), $p(x, y)$

- literal

  A literal is either an atom or its negation.

  - Example: $p(a, f(a, b))$, $\neg p(a, f(a, b))$

- (well-formed) formula

  - A well-formed (logic) formula is defined inductively as follows:

    - An atom is a formula.

    - If P and Q are formulas, then so are $\neg P$, $(P \wedge Q)$, $(P \vee Q)$, $(P \rightarrow Q)$, and $(P \leftrightarrow Q)$.

    - If x is a variable and P is a formula containing x, then $(\forall x P)$ and $(\exists x P)$ are formulas.

- clause
  - A clause is an expression of the following form:

$$\neg A_1 \vee \neg A_2 \vee \ldots \vee \neg A_n \vee B_1 \vee \ldots \vee B_m$$

where A_i and B_j are atoms.

  - The above expression can be written in the following equivalent form:

$$B_1 \vee \ldots \vee B_m \leftarrow A_1 \wedge \ldots \wedge A_n$$

consequent

antecedent

or

$$B_1, \ldots, B_m \leftarrow A_1, \ldots, A_n$$

- clause

A	B	$\neg A \vee B$
1	1	1
0	1	1
1	0	0
0	0	1

- Horn clause

  A Horn clause is a clause with the head containing only one positive atom.

$$B_m \leftarrow A_1, ..., A_n$$

Deductive Databases

- fact
  - A fact is a special Horn clause of the following form:

    $B \leftarrow$

    with all variables in B being instantiated. ($B \leftarrow$ can be simply written as B.)
- logic program

  A logic program is a set of Horn clauses.

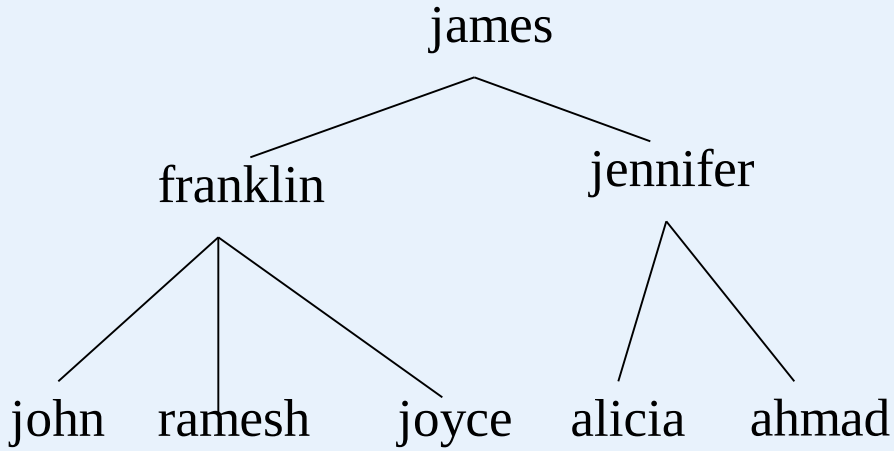
Deductive Databases

Facts can be considered as the data stored as relations in a relational database.

- Example (a logic program)

Facts:

supervise(franklin, john),
supervise(franklin, ramesh),
supervise(franklin, joyce)
supervise(james, franklin),
supervise(jennifer, alicia),
supervise(jennifer, ahmad),
supervise(james, jennifer).



Rules:

superior(X, Y) ← supervise(X, Y),
superior(X, Y) ← supervise(X, Z), superior(Z, Y),
subordinary(X, Y) ← superior(Y, X).

Deductive Databases

•Basic inference mechanism for logic programs

- interpretation of programs (rules + facts)

  There are two main alternatives for interpreting the theoretical meaning of rules:

  proof theoretic, and

  model theoretic interpretation

- proof theoretic interpretation

  1. The facts and rules are considered to be true statements, or *axioms*.

     facts - ground axioms

     rules - deductive axioms

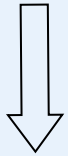
  2. The deductive axioms are used to construct proofs that derive new facts from existing facts.

Deductive Databases

- Example:

1. superior(X, Y) ← supervise(X, Y). (rule 1)
2. superior(X, Y) ← supervise(X, Z), superior (Z, Y). (rule 2)

3. supervise(jennifer, ahmad). (ground axiom, given)
4. supervise(james, jennifer). (ground axiom, given)



5. superior(jennifer, ahmad). (apply rule 1 on 3)
6. superior(james, ahmad). (apply rule 2 on 4 and 5)

- model theoretic interpretation
  1. Given a finite or an infinite domain of constant values, assign to each predicate in the program every possible combination of values as arguments.
  2. All the instantiated predicates contitute a *Herbrand* base.
  3. An interpretation is a subset of the Herbrand base.
  4. In the Herbrand base, each instantiated predicate evaluates to *true* or *false* in terms of the given facts and rules.
  5. An interpretation is called a **model** for a specific set of rules and the corresponding facts if those rules are always *true* under that interpretation.
  6. A model is a minimal model for a set of rules and facts if we cannot change any element in the model from *true* to *false* and still get a model for these rules and facts.

Deductive Databases

- Example:

1. superior(X, Y) ← supervise(X, Y). (rule 1)
2. superior(X, Y) ← supervise(X, Z), superior(Z, Y). (rule 2)

known facts:

supervise(franklin, john), supervise(franklin, ramesh),
supervise(franklin, joyce), supervise(james, franklin),
supervise(jennifer, alicia), supervise(jennifer, ahmad),
supervise(james, jennifer).
For all other possible (X, Y) combinations supervise(X, Y) is *false*.

domain = {james, franklin, john, ramesh, joyce, jennifer, alicia, ahmad}

Deductive Databases

Interpretation - model - minimal model

known facts:

supervise(franklin, john), supervise(franklin, ramesh),
supervise(franklin, joyce), supervise(james, franklin),
supervise(jennifer, alicia), supervise(jennifer, ahmad),
supervise(james, jennifer).
For all other possible (X, Y) combinations supervise(X, Y) is *false*.

derived facts:

superior(franklin, john), superior(franklin, ramesh),
superior(franklin, joyce), superior(jennifer, alicia),
superior(jennifer, ahmad), superior(james, franklin),
superior(james, jennifer), superior(james, john),
superior(james, ramesh), superior(james, joyce),
superior(james, alicia), superior(james, ahmad).
For all other possible (X, Y) combinations superior(X, Y) is *false*.

Deductive Databases

The above interpretation is also a model for the rules (1) and (2) since each of them evaluates always to *true* under the interpretation. For example,

superior(X, Y) ← supervise(X, Y)

superior(franklin, john) ← supervise(franklin, john) is *true*.
superior(franklin, ramesh) ← supervise(franklin, ramesh) is *true*.
... …

superior(X, Y) ← supervise(X, Z), superior(Z, Y)

superior(james, ramesh) ← supervise(james, franklin),
superior (franklin, ramesh) is *true*.
superior(james, alicia) ← supervise(james, jennifer),
superior (jennifer, alicia) is *true*.

The model is also the minimal model for the rule (1) and (2) and the corresponding facts since eliminating any element from the model will make some facts or instatiated rules evaluate to *false*.

- Inference mechanism

  In general, there are two approaches to evaluating logical programs: bottom-up and top-down.

  - Bottom-up mechanism

    (also called forward chaining and bottom-up resolution)

    1. The inference engine starts with the facts and applies the rules to generate new facts. That is, the inference moves forward from the facts toward the goal.
    2. As facts are generated, they are checked against the query predicate goal for a match.

Deductive Databases

- Example
  query goal: superior(james, Y)?
  rules and facts are given as above.

1. Check whether any of the existing facts directly matches the query.
2. Apply the first rule to the existing facts to generate new facts.
3. Apply the second rule to the existing facts to generate new facts.
4. As each fact is gnerated, it is checked for a match of the the query goal.
5. Repeat step 1 - 4 until no more new facts can be found.

All the facts of the form: superior(james, a) are the answers.

- Example:

1. superior(X, Y) ← supervise(X, Y). (rule 1)
2. superior(X, Y) ← supervise(X, Z), superior(Z, Y). (rule 2)

known facts:

supervise(franklin, john), supervise(franklin, ramesh),
supervise(franklin, joyce), supervise(james, franklin),
supervise(jennifer, alicia), supervise(jennifer, ahmad),
supervise(james, jennifer).
For all other possible (X, Y) combinations supervise(X, Y) is *false*.

domain = {james, franklin, john, ramesh, joyce, jennifer, alicia, ahmad}

superior(james, Y)?

applying the first rule: superior(james, franklin), superior(james, jennifer)
Y = {franklin, jennifer}

applying the second rule: Y = {John, Joyce, Ramesh, alicia, ahmad}

Deductive Databases

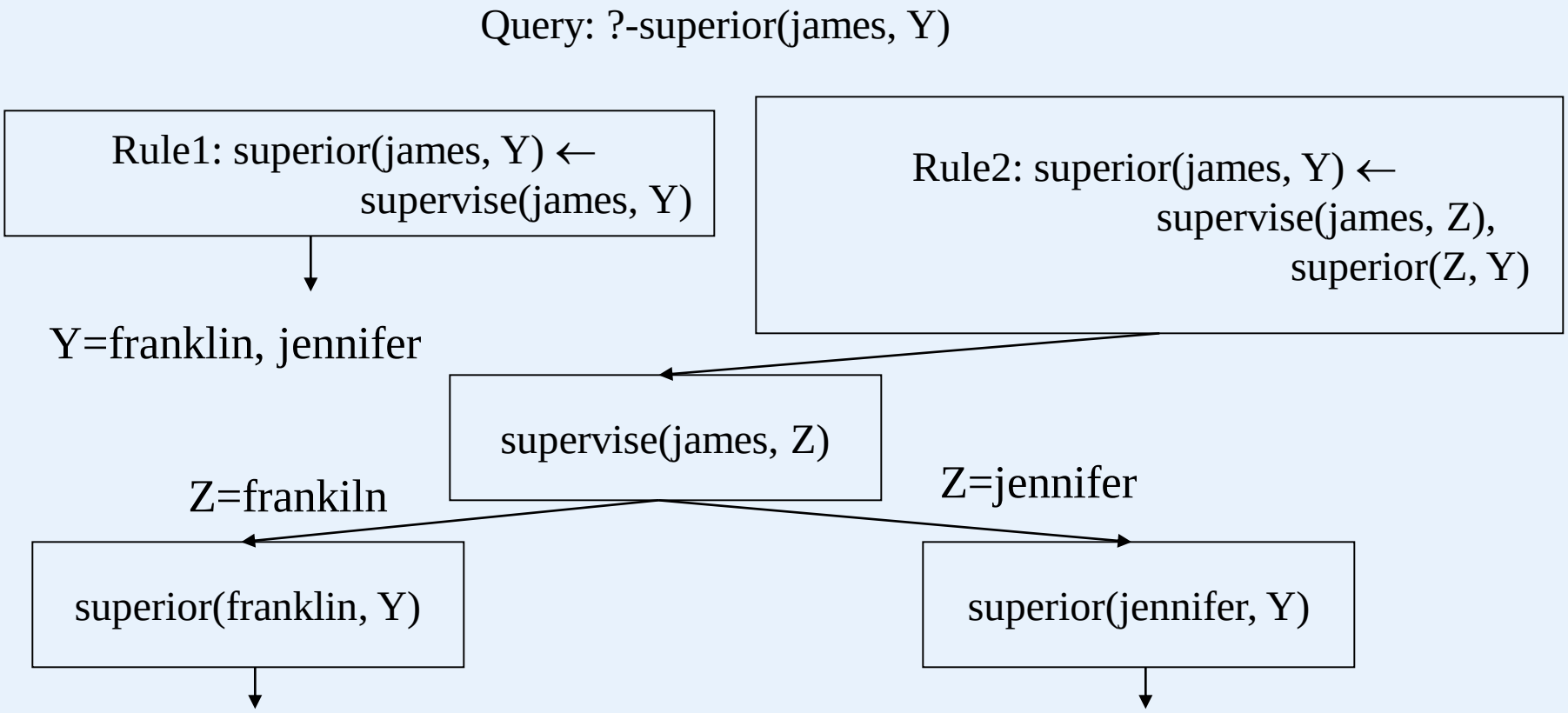
Top-down mechanism

(also called back chaining and top-down resolution)

1. The inference engine starts with the query goal and attempts to find matches to the variables that lead to valid facts in the database. That is, the inference moves backward from the intended goal to determine facts that would satisfy the goal.
2. During the course, the rules are used to generate subgoals. The matching of these subgoals will lead to the match of the intended goal.

Deductive Databases

- Example
  query goal: ?-superior(james, Y)
  rules and facts are given as above.



Deductive Databases

Rule1: superior(franklin, Y) ← supervise(franklin, Y)

↓

Y= john, ramesh, joyce

Rule1: superior(jennifer, Y) ← supervise(jennifer, Y)

↓

Y= alicia, ahmad

- **Datalog programs and their evaluation**

1. A Datalog program is a logic program.
2. In a Datalog program, each predicate contains no function symbols.
3. A Datalog program normally contains two kinds of predicates: fact-based predicates and rule-based predicates.

   fact-based predicates are defined by listing all the combinations of values that make the predicate true.

   Rule-based predicates are defined to be the head of one or more Datalog rules. They correspond to virtual relations whose contents can be inferred by the inference engine.